

✓ Downloading Netflix data in csv format

```
!wget "https://drive.google.com/uc?export=download&id=1-qDO7oNwzQn0RV44YtpqWdYS4SO3GkQg" -O netflix.csv
```

```
#Importing essencial libraries
```

```
import pandas as pd
import matplotlib.pyplot as plt
import numpy as np
import seaborn as sns
```

```
##Read excel file*
```

```
data=pd.read_csv('netflix.csv')
```

```
data.head(2)
```

	show_id	type	title	director	cast	country	date_added	release_year	rating	duration	listed_in	description
0	s1	Movie	Dick Johnson Is Dead	Kirsten Johnson	NaN	United States	September 25, 2021	2020	PG-13	90 min	Documentaries	As her father nears the end of his life, filmm...

Next steps:

[Generate code with data](#)
[View recommended plots](#)
[New interactive sheet](#)

Observations on the shape of data, data types of all the attributes, conversion of

✓ categorical attributes to 'category' (If required), missing value detection, statistical summary (10 Points)

```
data.head(2)
```

	show_id	type	title	director	cast	country	date_added	release_year	rating	duration	listed_in	description
0	s1	Movie	Dick Johnson Is Dead	Kirsten Johnson	NaN	United States	September 25, 2021	2020	PG-13	90 min	Documentaries	As her father nears the end of his life, filmm...

- Find total rows and columns

```
data.shape
```

```
(8807, 12)
```

- Find the total columns name

```
data.columns
```

```
Index(['show_id', 'type', 'title', 'director', 'cast', 'country', 'date_added',  
       'release_year', 'rating', 'duration', 'listed_in', 'description'],  
      dtype='object')
```

- Finding the total number null values column wise in dataset

```
data.isnull().sum()
```

	0
show_id	0
type	0
title	0
director	2634
cast	825
country	831
date_added	10
release_year	0
rating	4
duration	3
listed_in	0
description	0

dtype: int64

- in above table highest number of null values present in director column
- Making another copy of dataset so that whatever changes we do it will not affect on original Dataset

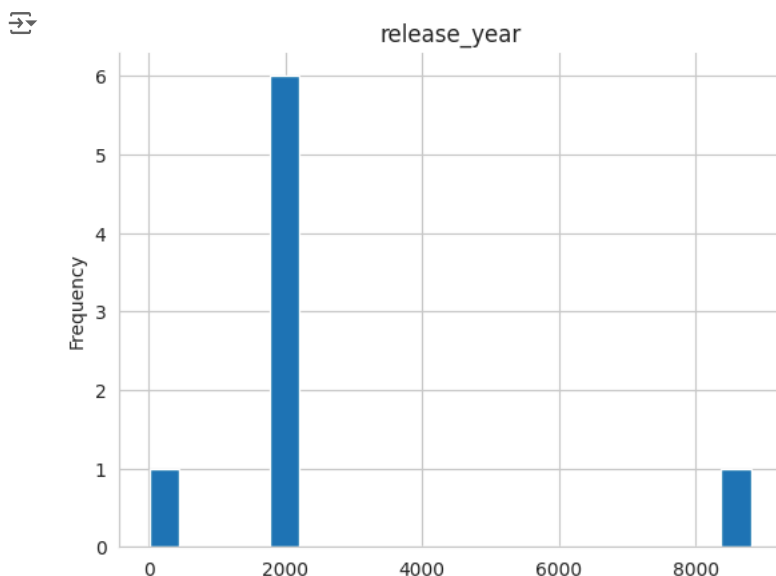
```
df=data.copy()
```

- Statistical analysis of given data

```
df.describe()
```

	release_year
count	8807.000000
mean	2014.180198
std	8.819312
min	1925.000000
25%	2013.000000
50%	2017.000000
75%	2019.000000
max	2021.000000

```
from matplotlib import pyplot as plt
_df_0['release_year'].plot(kind='hist', bins=20, title='release_year')
plt.gca().spines[['top', 'right']].set_visible(False)
```



Converting Categorical attributes to category

df.columns

```
Index(['show_id', 'type', 'title', 'director', 'cast', 'country', 'date_added',
      'release_year', 'rating', 'duration', 'listed_in', 'description'],
      dtype=object)
```

- Before conversion of datatype of categorical attributes

df.info()

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 8807 entries, 0 to 8806
Data columns (total 12 columns):
#   Column      Non-Null Count  Dtype
---  -
0  show_id      8807 non-null   object
1  type         8807 non-null   object
2  title        8807 non-null   object
3  director     6173 non-null   object
4  cast         7982 non-null   object
5  country      7976 non-null   object
6  date_added   8797 non-null   object
7  release_year 8807 non-null   int64
8  rating       8803 non-null   object
9  duration     8804 non-null   object
10 listed_in   8807 non-null   object
11 description 8807 non-null   object
dtypes: int64(1), object(11)
memory usage: 825.8+ KB
```

- Converting object type to Category for categorical columns

```
df['type']=df['type'].astype('category')
df['country']=df['country'].astype('category')
df['rating']=df['rating'].astype('category')
df['duration']=df['duration'].astype('category')
df['release_year']=df['release_year'].astype('category')
df['director']=df['director'].astype('category')
```

- After conversion of datatype from object type to Category type of categorical attributes

df.info()

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 8807 entries, 0 to 8806
Data columns (total 12 columns):
#   Column      Non-Null Count  Dtype
---  -
0  show_id      8807 non-null   object
1  type         8807 non-null   category
2  title        8807 non-null   object
3  director     6173 non-null   category
4  cast         7982 non-null   object
5  country      7976 non-null   category
6  date_added   8797 non-null   object
7  release_year 8807 non-null   category
8  rating       8803 non-null   category
9  duration     8804 non-null   category
10 listed_in   8807 non-null   object
11 description 8807 non-null   object
dtypes: category(6), object(6)
memory usage: 690.0+ KB
```

✓ Pre-Processing Data

✓ Un-nesting of the data in columns like Cast(Actor), Director, Country, listed_in

- Converting the comma separated values into list in Cast column

```
df['cast']=df['cast'].str.split(',')
```

```
df.head(2)
```

	show_id	type	title	director	cast	country	date_added	release_year	rating	duration	listed_in	description
0	s1	Movie	Dick Johnson Is Dead	Kirsten Johnson	NaN	United States	September 25, 2021	2020	PG-13	90 min	Documentaries	As her father nears the end of his life, filmm...
					[Ama Qamata,						International TV	After crossing

- in the above data we can see Cast names are separated into list.

- Now explode the data into seperate rows

```
df=df.explode('cast')
```

```
df.head(2)
```

	show_id	type	title	director	cast	country	date_added	release_year	rating	duration	listed_in	description
0	s1	Movie	Dick Johnson Is Dead	Kirsten Johnson	NaN	United States	September 25, 2021	2020	PG-13	90 min	Documentaries	As her father nears the end of his life, filmm...
											International TV	After crossing

- We can see the cast name Ama Qamata is moved to seperate row

- Same we need to perform with listed_in column

```
df['listed_in']=df['listed_in'].str.split(',')
```

```
df.head(2)
```

	show_id	type	title	director	cast	country	date_added	release_year	rating	duration	listed_in	description
0	s1	Movie	Dick Johnson Is Dead	Kirsten Johnson	NaN	United States	September 25, 2021	2020	PG-13	90 min	[Documentaries]	As her father nears the end of his life, filmm...
											[International TV]	After crossing

```
df=df.explode('listed_in')
```

```
df.head(2)
```

	show_id	type	title	director	cast	country	date_added	release_year	rating	duration	listed_in	description
0	s1	Movie	Dick Johnson Is Dead	Kirsten Johnson	NaN	United States	September 25, 2021	2020	PG-13	90 min	Documentaries	As her father nears the end of his life, filmm...
												After crossing

- Un-nesting of country column

```
df['country']=df['country'].str.split(',')
```

```
df.head(2)
```

	show_id	type	title	director	cast	country	date_added	release_year	rating	duration	listed_in	description
0	s1	Movie	Dick Johnson Is Dead	Kirsten Johnson	NaN	[United States]	September 25, 2021	2020	PG-13	90 min	Documentaries	As her father nears the end of his life, filmm...
												After crossing

```
df=df.explode('country')
```

- Un-nesting of Director column

```
df['director']=df['director'].str.split(',')
```

```
df=df.explode('director')
```

```
df.tail(2)
```



	show_id	type	title	director	cast	country	date_added	release_year	rating	duration	listed_in	description
8806	s8807	Movie	Zubaan	Mozez Singh	Chittaranjan Tripathy	India	March 2, 2019	2015	TV-14	111 min	International Movies	A scrappy but poor boy worms his way into a ty...

✓ Handling Hull values

- Cheking the number of null values column wise

```
df.isnull().sum()
```

	0
show_id	0
type	0
title	0
director	50643
cast	2149
country	11897
date_added	158
release_year	0
rating	67
duration	3
listed_in	0
description	0
dtype:	int64

```
df.fillna({'director':df.loc[df['type']=='TV Show']['director'].mode()[0]},inplace=True)
```

```
df.fillna({'director':df.loc[df['type']=='Movie']['director'].mode()[0]},inplace=True)
```

```
df.fillna({'cast':df.loc[df['type']=='TV Show']['cast'].mode()[0]},inplace=True)
```

```
df.fillna({'cast':df.loc[df['type']=='Movie']['cast'].mode()[0]},inplace=True)
```

```
df.fillna({'country':df.loc[df['type']=='Movie']['country'].mode()[0]},inplace=True)
```

```
df.fillna({'country':df.loc[df['type']=='Movie']['country'].mode()[0]},inplace=True)
```

```
df.isnull().sum()
```



	0
show_id	0
type	0
title	0
director	0
cast	0
country	0
date_added	158
release_year	0
rating	67
duration	3
listed_in	0
description	0

dtype: int64

- Filling null values in date column(date_added)

df.head(1)



	show_id	type	title	director	cast	country	date_added	release_year	rating	duration	listed_in	description
0	s1	Movie	Dick Johnson Is Dead	Kirsten Johnson	David Attenborough	United States	September 25, 2021	2020	PG-13	90 min	Documentaries	As her father nears the end of his

Filling appropriate values in day,month and year columns(launch_day,launch_month,launch_year)

- First converting the data type of date column(date_added) from object to datetime64

```
df['date_added']=pd.to_datetime(df['date_added'].astype('datetime64[ns]'))
df['date_added'].info()
```



```
<class 'pandas.core.series.Series'>
Index: 202065 entries, 0 to 8806
Series name: date_added
Non-Null Count  Dtype
-----
201907 non-null  datetime64[ns]
dtypes: datetime64[ns](1)
memory usage: 3.1 MB
```

- Separating the date column into launch_day,launch_month,launch_year

```
df['launch_day']=pd.to_numeric(df['date_added'].dt.day)
df['launch_month']=pd.to_numeric(df['date_added'].dt.month)
df['launch_year']=pd.to_numeric(df['date_added'].dt.year)
```

```
df['launch_day']=df['launch_day'].fillna(df['launch_day'].mode()[0])
df['launch_month']=df['launch_month'].fillna(df['launch_month'].mode()[0])
df['launch_year']=df['launch_year'].fillna(df['launch_year'].mode()[0])
```

Converting data type from float to int so that value 2013.0 will become 2013

Tip: We are not able to concat the int column value so that I have again converted from int32 to object str.

```
df['launch_year']=df['launch_year'].astype(np.int64).astype(str)
df['launch_day']=df['launch_day'].astype(np.int64).astype(str)
df['launch_month']=df['launch_month'].astype(np.int64).astype(str)
```

- Concatenating the launch_day,launch_month,launch_year columns in YYYY-MM-DD format and filled the null date into **date_added** column with the help of concatenated columns

```
df['date_added']=df['date_added'].fillna(df['launch_year']+'-'+df['launch_month']+'-'+df['launch_day'])
df.isnull().sum()
```

```
↵
0
show_id    0
type       0
title      0
director   0
cast       0
country    0
date_added 0
release_year 0
rating     67
duration   3
listed_in  0
description 0
launch_day 0
launch_month 0
launch_year 0
```

dtype: int64

- Now we have filled the all null values for all columns.

• Filling Null values in Rating column

```
df.loc[df['rating'].isnull(),['show_id','rating','country','type','director']]
```

```
↵
   show_id  rating  country  type  director
5989  s5990   NaN  United States  Movie  Noam Murro
5989  s5990   NaN  United States  Movie  Noam Murro
6827  s6828   NaN    Japan  TV Show  Noam Murro
6827  s6828   NaN    Japan  TV Show  Noam Murro
6827  s6828   NaN    Japan  TV Show  Noam Murro
...      ...    ...      ...    ...      ...
7537  s7538   NaN    Italy  Movie  Alessandro Pepe
7537  s7538   NaN    Italy  Movie  Alessandro Pepe
7537  s7538   NaN    Italy  Movie  Alessandro Pepe
7537  s7538   NaN    Italy  Movie  Alessandro Pepe
7537  s7538   NaN    Italy  Movie  Alessandro Pepe
```

67 rows × 5 columns

- To fill the NaN rating I found the rating mode value for specific country, Type and Director to fill NaN for specific county

```
df.loc[(df['country']=='Japan') & (df['type']=='TV Show') & (df['director'].notnull())]['rating'].mode()
```

```
↵
rating
0    TV-14

dtype: category
```

```
df.loc[(df['country']=='Japan') & (df['type']=='TV Show') & (df['rating'].isnull()), 'rating'] = df.loc[(df['country']=='Japan') & (df['type']=='TV Show') & (df['director'].notnull())]
```

```
df.loc[(df['country']=='Japan') & (df['type']=='TV Show') & (df['show_id']=='s6828')].head(2)
```

	show_id	type	title	director	cast	country	date_added	release_year	rating	duration	listed_in	description	launc	
	6827	s6828	TV Show	Gargantia on the Verdurous Planet	Noam Murro Kaito Ishikawa	Japan	2016-12-01	2013	TV-14	1 Season	Anime Series	After falling through a wormhole, a space-dwel...		
	6827	s6828	TV	Gargantia on the	Noam	Kaito	Japan	2016-12-01	2013	TV-14	1	International	After falling through a wormhole. a	

```
df.fillna({'rating':df.loc[(df['country']=='Australia') & (df['type']=='TV Show') & (df['director'].notnull())]['rating'].mode()[0]},inplace=True)
```

```
df['duration'].value_counts().sort_values(ascending=False).reset_index()['duration'][0]
```

```
'1 Season'
```

```
df.fillna({'duration':df['duration'].value_counts().sort_values(ascending=False).reset_index()['duration'][0]},inplace=True)
```

```
df.isnull().sum()
```

```

0
show_id    0
type       0
title      0
director   0
cast       0
country    0
date_added 0
release_year 0
rating     0
duration   0
listed_in  0
description 0
launch_day 0
launch_month 0
launch_year 0

dtype: int64

```

- Now we have filled the Null/NaN values with appropriate value

4. Comparison of tv shows vs. movies for India and Non-India countries

- Separating the data for India and Non-India

```
india_nf_show=df.loc[(df['country']=='India'),['show_id','type','title','director','country','rating','duration']]
nonindia_nf_show=df.loc[(df['country']!='India'),['show_id','type','title','director','country','rating','duration']]
```

Total number of Movies and TV Show from India and Non-India

```
non_india_d=nonindia_nf_show.groupby('type')['show_id'].count().reset_index()
non_india_d
```



```
<ipython-input-311-78a0bc5ffd8b>:1: FutureWarning: The default of observed=False is deprecated and will be changed to True in a future version of pandas. Pa
non_india_d=nonindia_nf_show.groupby('type')['show_id'].count().reset_index()
```

	type	show_id
0	Movie	125156
1	TV Show	54770

```
india_d=india_nf_show.groupby('type')['show_id'].count().reset_index()
india_d
```

```
<ipython-input-312-d093f5f16b84>:1: FutureWarning: The default of observed=False is deprecated and will be changed to True in a future version of pandas. Pa
india_d=india_nf_show.groupby('type')['show_id'].count().reset_index()
```

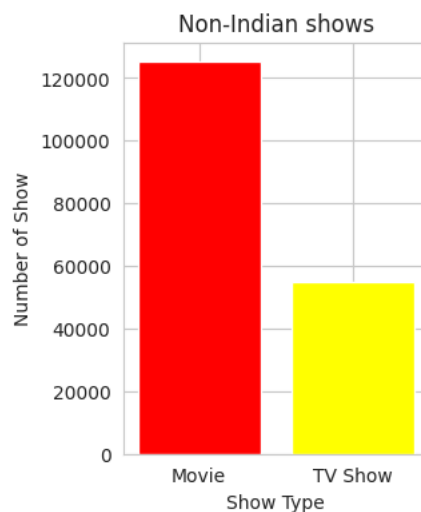
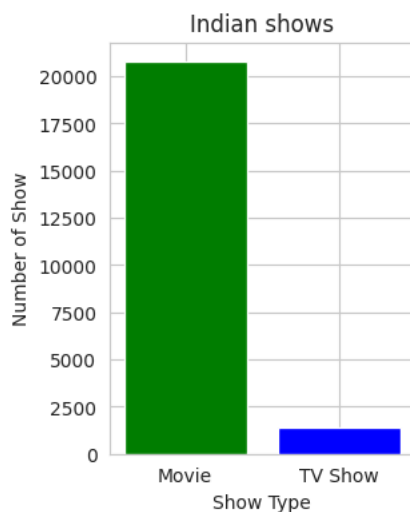
	type	show_id
0	Movie	20761
1	TV Show	1378

✓ Comparison of tv shows vs. movies from India and Non-India in a graphical format.

```
plt.figure(figsize=(10,4))
plt.subplot(1,3,1)
c=['green','blue']
plt.bar('type','show_id',data=india_d,color=c)
plt.xlabel('Show Type')
plt.ylabel('Number of Show')
plt.title('Indian shows')
plt.plot(legend=True)
```

```
plt.subplot(1,3,3)
c=['red','yellow']
plt.bar('type','show_id',data=non_india_d,color=c)
plt.xlabel('Show Type')
plt.ylabel('Number of Show')
plt.title('Non-Indian shows')
plt.plot(legend=True)
```

```
↳ []
```



- In the above snapshot we can see that the number of movies are more than number of TV Shows for India and Non-India country

✓ Top 10 highest count of tv shows released/produced country wise

✓ Top 10 highest count of TV Shows released/produced country wise

```
tv_shows=df.loc[(df['type']=='TV Show'),['type','country']].groupby('country')['type'].count()
tv_shows=tv_shows.sort_values(ascending=False).reset_index()
tv_shows=tv_shows.head(10)
tv_shows
```



	country	type
0	United States	17250
1	Japan	4673
2	United Kingdom	4078
3	South Korea	3648
4	United States	1981
5	Taiwan	1719
6	Mexico	1683
7	Spain	1647
8	Canada	1467
9	India	1378

✓ Top 10 highest count of Movies released/produced country wise

```
Movie=df.loc[(df['type']=='Movie'),['type','country']].groupby('country')['type'].count()
Movie=Movie.sort_values(ascending=False).reset_index()
Movie=Movie.head(10)
Movie
```

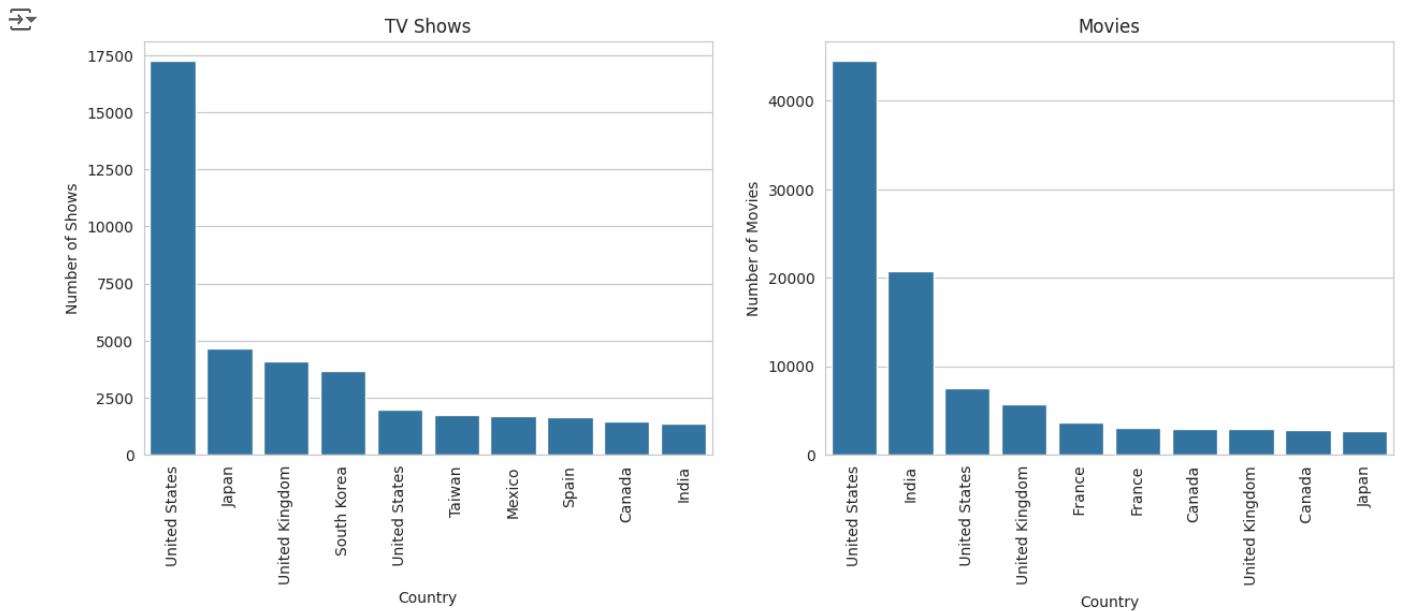


	country	type
0	United States	44515
1	India	20761
2	United States	7501
3	United Kingdom	5655
4	France	3592
5	France	3015
6	Canada	2928
7	United Kingdom	2925
8	Canada	2810
9	Japan	2644

✓ Top 10 movies and TV Shows released/produced from different countries Visualization

```
plt.figure(figsize=(15,5))
plt.subplot(1,2,1)
sns.barplot(x=tv_shows['country'],y=tv_shows['type'])
plt.xticks(rotation=90)
plt.xlabel('Country')
plt.ylabel('Number of Shows')
plt.title('TV Shows')
```

```
plt.subplot(1,2,2)
sns.barplot(x=Movie['country'],y=Movie['type'])
plt.xticks(rotation=90)
plt.xlabel('Country')
plt.ylabel('Number of Movies')
plt.title('Movies')
plt.show()
```



- on above snapshot we can see that highest number of TV shows are launched from US and Null Countries
- and other side highest number of Movies are launched from US and India Countries

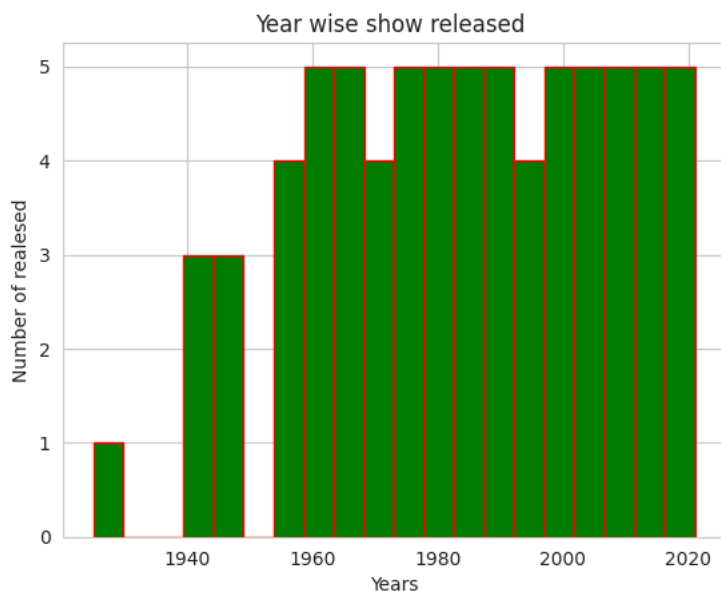
Above chart Display total movie and TV show released year wise

```
year=pd.Series(df['release__year'])
year=year.unique()
year
```

```
[2020, 2021, 1993, 2018, 1996, ..., 1942, 1955, 1944, 1947, 1943]
Length: 74
Categories (74, int64): [1925, 1942, 1943, 1944, ..., 2018, 2019, 2020, 2021]
```

- we found the unique years of released_year column
- As per the chart below we can see that most number of movies and TV shows released between 2015-2020

```
plt.hist(year,edgecolor='red',bins=20,color='green')
plt.xlabel('Years')
plt.ylabel('Number of realesed')
plt.title('Year wise show released ')
plt.show()
```



```
df.head(2)
```



show_id	type	title	director	cast	country	date_added	release_year	rating	duration	listed_in	description	lau
0	s1	Movie	Dick Johnson Is Dead	Kirsten Johnson	David Attenborough	United States	2021-09-25	2020	PG-13	90 min	Documentaries	As her father nears the end of his life, filmm... After

- Find the horror movies.

```
df.loc[df['description'].str.contains('horror',case=False), 'description'].head(2)
```



	description
211	The horror continues when Ragini's video goes ...
211	The horror continues when Ragini's video goes ...

dtype: object

✓ Highest number of Horror shows and Comedy Shows

- 1.Top Countries which has total number of horror movies/shows released from in which country.

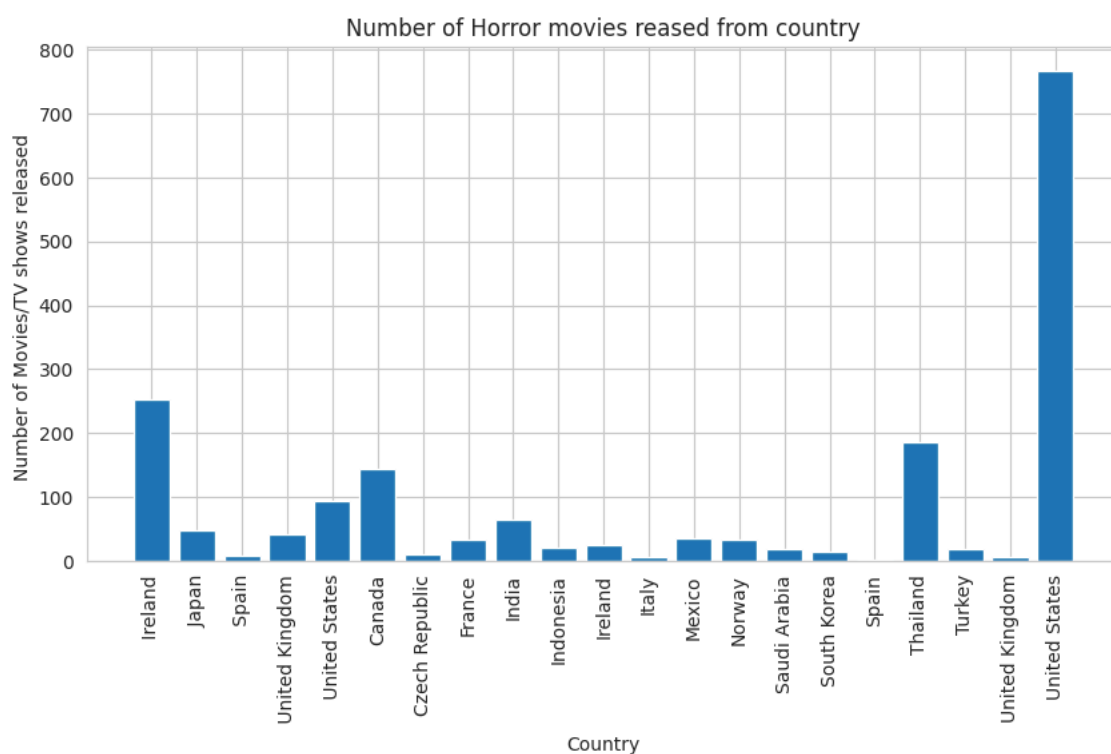
```
horror_show=df.loc[(df['description'].str.contains('horror',case=False)),['description','country']].groupby('country')['description'].count().reset_index()
horror_show.head(10)
```



	country	description
0	Ireland	252
1	Japan	48
2	Spain	8
3	United Kingdom	41
4	United States	94
5	Canada	144
6	Czech Republic	10
7	France	33
8	India	64
9	Indonesia	20
10	Ireland	24
11	Italy	7
12	Mexico	36
13	Norway	33
14	Saudi Arabia	18
15	South Korea	15
16	Spain	2
17	Thailand	185
18	Turkey	18
19	United Kingdom	7
20	United States	766

- In the above table we can see that the largest number of horror movies/shows released from US country on Netflix, and second largest horror movie released from Canada.

```
plt.figure(figsize=(10,5))
plt.bar('country','description', data=horror_show)
plt.xticks(rotation=90)
plt.xlabel('Country')
plt.ylabel('Number of Movies/TV shows released ')
plt.title('Number of Horror movies reased from country')
plt.show()
```



- In the above graph we can see that the largest number of horror movies/shows released from US country on Netflix, and second largest horror movie released from Canada.

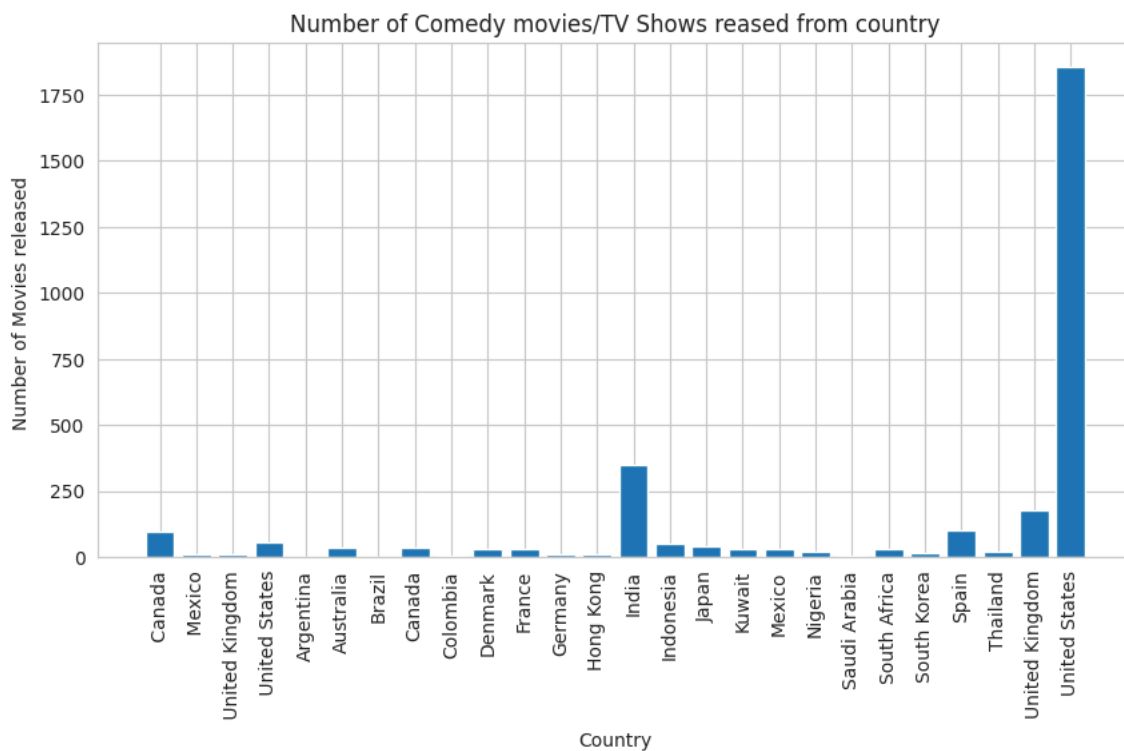
✓ Finding the Comedy Shows

```
comedy_show=df.loc[(df['description'].str.contains('comedy',case=False)),['description','country']].groupby('country')['description'].count().reset_index()
comedy_show
```



	country	description
0	Canada	96
1	Mexico	8
2	United Kingdom	12
3	United States	55
4	Argentina	2
5	Australia	37
6	Brazil	1
7	Canada	33
8	Colombia	3
9	Denmark	30
10	France	31
11	Germany	8
12	Hong Kong	12
13	India	350
14	Indonesia	48
15	Japan	41
16	Kuwait	30
17	Mexico	31
18	Nigeria	18
19	Saudi Arabia	4
20	South Africa	30
21	South Korea	15
22	Spain	103
23	Thailand	21
24	United Kingdom	176
25	United States	1855

```
plt.figure(figsize=(10,5))
plt.bar('country','description', data=comedy_show)
plt.xticks(rotation=90)
plt.xlabel('Country')
plt.ylabel('Number of Movies released ')
plt.title('Number of Comedy movies/TV Shows reased from country')
plt.show()
```



- Above bar chart display the number of comedy shows launched by different countries
- And US has the highest number of comedy shows on Netflix
- Top 10 contries which produced or released comedy movies/TV show on Netflix

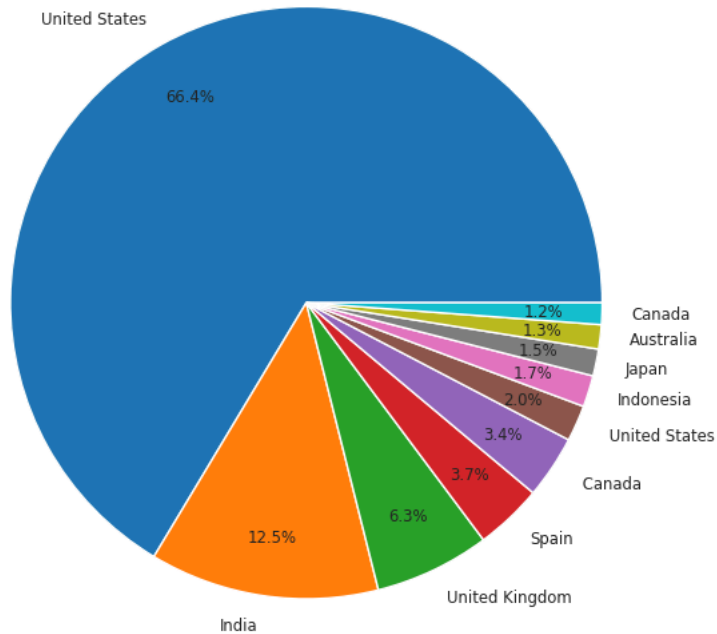
```
top10_comedy_show=comedy_show.sort_values('description',ascending=False)
top10_comedy_show=top10_comedy_show.reset_index().head(10)
top10_comedy_show
```



	index	country	description
0	25	United States	1855
1	13	India	350
2	24	United Kingdom	176
3	22	Spain	103
4	0	Canada	96
5	3	United States	55
6	14	Indonesia	48
7	15	Japan	41
8	5	Australia	37
9	7	Canada	33

- As per the pie chart we can clearly see that the highest percentage of comedy movies is from of US

```
plt.pie(top10_comedy_show['description'],labels=top10_comedy_show.country,autopct='%1.1f%%', pctdistance=0.8,textprops={'size': 'smaller'}, radius=1.5)
plt.xticks(rotation=90)
plt.show()
```



- As we can see the highest percentage country is US which produced or released highest number of movies/TV shows on Netflix followed by India

✓ Analysis of top 10 Actors and Directors of different types of TV Shows and Movies.

✓ a. Identify the top 10 Actors who have appeared in most movies or TV shows.

Top 10 actors of movies

```
actor=df.loc[(df['type']=='Movie'),['cast','title']].drop_duplicates().groupby('cast')['title'].count().reset_index()
top10_movie_actor=actor.sort_values('title',ascending=False)
top10_movie_actor.head(10)
```



	cast	title
24942	David Attenborough	481
1946	Anupam Kher	38
19235	Rupa Bhimani	27
16781	Om Puri	27
27291	Shah Rukh Khan	26
3109	Boman Irani	25
17025	Paresh Rawal	25
11219	Julie Teiwani	24
24247	Akshay Kumar	23
18089	Rajesh Kava	21

Top 10 Actors of TV Shows

```
actor=df.loc[(df['type']=='TV Show'),['cast','title']].drop_duplicates().groupby('cast')['title'].count().reset_index()
top10_TVShow_actor=actor.sort_values('title',ascending=False)
top10_TVShow_actor.head(10)
```




	cast	title
13923	David Attenborough	364
11974	Takahiro Sakurai	24
6136	Junichi Suwabe	17
222	Ai Kayano	17
13204	Yuki Kaji	17
2573	Daisuke Ono	14
13125	Yoshimasa Hosoya	13
11989	Takehito Koyasu	13
13188	Yuichi Nakamura	13
12410	Tomokazu Sugita	12

-Assigning values to x and y to generate plot

```
top10_TVShow_actor=top10_TVShow_actor.loc[(top10_TVShow_actor['cast']!= 'Unknown cast'),['cast','title']]
x=top10_TVShow_actor['cast'].head(10)
y=top10_TVShow_actor['title'].head(10)

top10_movie_actor=top10_movie_actor.loc[(top10_movie_actor['cast']!= 'Unknown cast'),['cast','title']]
a=top10_movie_actor['cast'].head(10)
b=top10_movie_actor['title'].head(10)
```

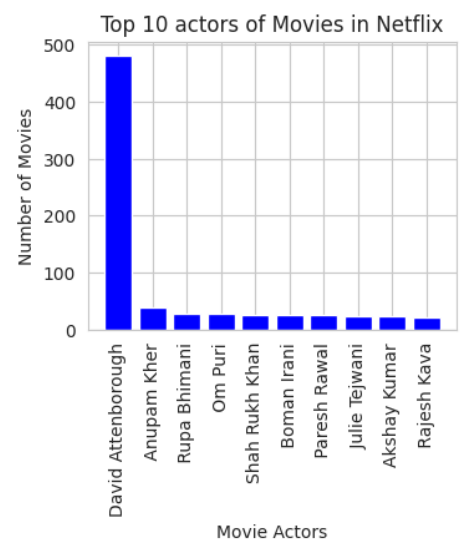
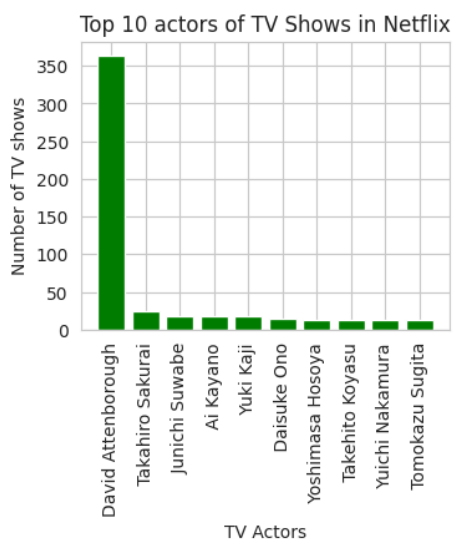
- Visual representation of top 10 Actors of TV Shows and Movies in Netflix

=> Please ignore the Unknow Cast for this case study as per condition

```
plt.figure(figsize=(13,3))
plt.subplot(1,3,1)
plt.bar(x,y,color='green')
plt.xlabel('TV Actors')
plt.ylabel('Number of TV shows')
plt.title('Top 10 actors of TV Shows in Netflix')
plt.xticks(rotation=90)
```

```
plt.subplot(1,3,3)
plt.bar(a,b,color='blue')
plt.xlabel('Movie Actors')
plt.ylabel('Number of Movies')
plt.title('Top 10 actors of Movies in Netflix')
plt.xticks(rotation=90)
```

```
plt.show()
```



- ✓ b. Identify the top 10 directors who have appeared in most movies or TV shows.

- **Top 10 Directors of TV Shows which have directed most number of TV Shows on Netflix platform.**

```
df.head(2)
```

	show_id	type	title	director	cast	country	date_added	release_year	rating	duration	listed_in	description	lau
0	s1	Movie	Dick Johnson Is Dead	Kirsten Johnson	David Attenborough	United States	2021-09-25	2020	PG-13	90 min	Documentaries	As her father nears the end of his life, filmm...	After

```
director=df.loc[(df['type']=='TV Show'),['director','title']].drop_duplicates().groupby('director')['title'].count().reset_index()
top10_TVShow_director=director.sort_values('title',ascending=False)
top10_TVShow_director.head(10)
```

	director	title
227	Noam Murro	2447
187	Ken Burns	3
86	Alastair Fothergill	3
173	Joe Berlinger	2
255	Rob Seidenglanz	2
153	Hsu Fu-chun	2
265	Shin Won-ho	2
24	Gautham Vasudev Menon	2
183	Jung-ah Im	2
155	Iginio Straffi	2

- **Top 10 Directors of Movies which have directed most number of Movies on Netflix platform.**

```
director=df.loc[(df['type']=='Movie'),['director','title']].drop_duplicates().groupby('director')['title'].count().reset_index()
top10_movies_director=director.sort_values('title',ascending=False)
top10_movies_director.head(10)
```

	director	title
3505	Noam Murro	189
3818	Rajiv Chilaka	22
234	Jan Suter	18
3865	Raúl Campos	18
4429	Suhas Kadav	16
2308	Jay Karas	15
3067	Marcus Raboy	15
1289	Cathy Garcia-Molina	13
4840	Youssef Chahine	12
3135	Martin Scorsese	12

```
top10_TVShow_director=top10_TVShow_director.loc[(top10_TVShow_director['director']!='Unknown director'),['director','title']]
x=top10_TVShow_director['director'].head(10)
y=top10_TVShow_director['title'].head(10)
```

```
top10_movies_director=top10_movies_director.loc[(top10_movies_director['director']!='Unknown director'),['director','title']]
a=top10_movies_director['director'].head(10)
b=top10_movies_director['title'].head(10)
```

Below chart represent the top 10 Directors of TV Shows and Movies in Netflix

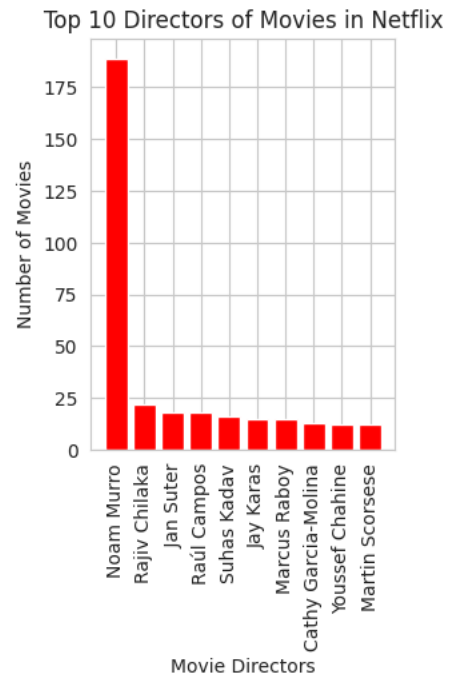
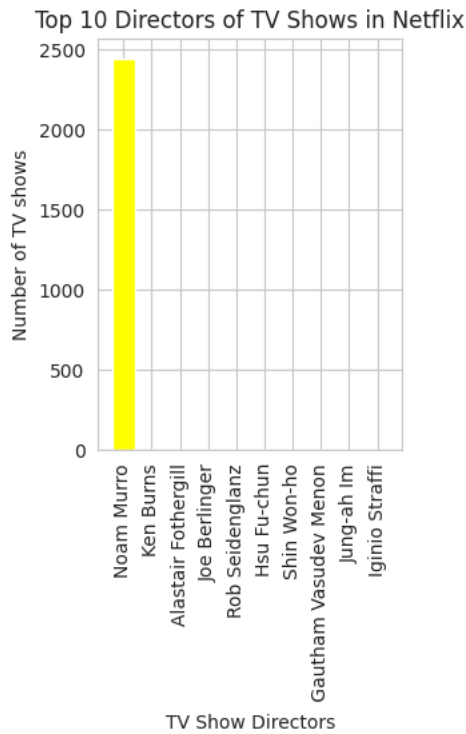
=> Please ignore the Unknow director for this case study as per condition

```
plt.figure(figsize=(10,4))
```

```
plt.subplot(1,3,1)
plt.bar(x,y,color='yellow')
plt.xlabel('TV Show Directors')
plt.ylabel('Number of TV shows')
plt.title('Top 10 Directors of TV Shows in Netflix')
plt.xticks(rotation=90)
```

```
plt.subplot(1,3,3)
plt.bar(a,b,color='r')
plt.xlabel('Movie Directors')
plt.ylabel('Number of Movies')
plt.title('Top 10 Directors of Movies in Netflix')
plt.xticks(rotation=90)
```

```
plt.show()
```



✓ What is the best time to launch a TV show?

- Converting "date_added" object column to datetime to fetch the weekday

```
df['date_added'] = df['date_added'].astype('datetime64[ns]')
df['date_added'].info()
```



```
<class 'pandas.core.series.Series'>
Index: 202065 entries, 0 to 8806
Series name: date_added
Non-Null Count  Dtype
-----
202065 non-null  datetime64[ns]
dtypes: datetime64[ns](1)
memory usage: 3.1 MB
```

Dropping the duplicate rows for the below mention columns to get accurate details of TV Shows and Movies

```
unique_show=df[['show_id','type','date_added','release_year','launch_day','launch_month','launch_year']].drop_duplicates().reset_index()
```

- Generating the new column launch_weekday to store the number of week of day.

```
unique_show['launch_weekday']=unique_show['date_added'].dt.weekday
unique_show.head(2)
```

- Separating the Movies data and TV Shows Data

```
movie_shows=unique_show.loc[unique_show['type']=='Movie']
tv_shows=unique_show.loc[unique_show['type']=='TV Show']
tv_shows.head()
```

→

	index	show_id	type	date_added	release_year	launch_day	launch_month	launch_year	launch_weekday
1	1	s2	TV Show	2021-09-24	2021	24	9	2021	4
2	2	s3	TV Show	2021-09-24	2021	24	9	2021	4
3	3	s4	TV Show	2021-09-24	2021	24	9	2021	4
4	4	s5	TV Show	2021-09-24	2021	24	9	2021	4
5	5	s6	TV Show	2021-09-24	2021	24	9	2021	4

✓ 1. Find which is the best week to release the Tv-show or the movie.

- Analysis for Movies
- **Movies:**
- We will going to find the best week day on which highest number of **Movies** *launched on Netflix*

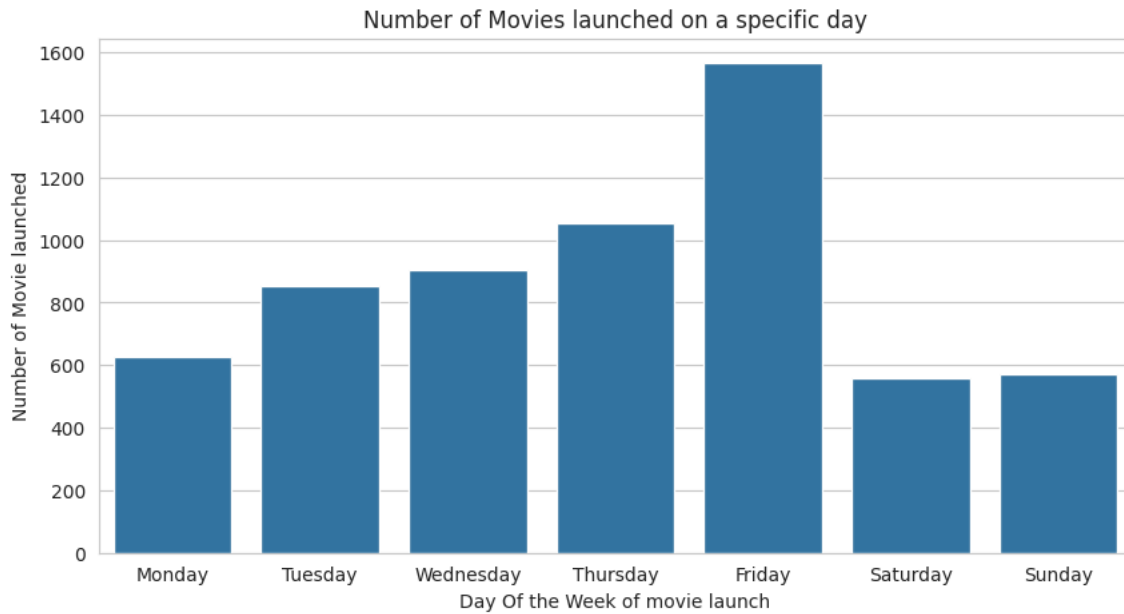
Count the number of Movies released on weekday wise

```
weekday_group=movie_shows.groupby('launch_weekday')['show_id'].count().reset_index()
days={'Dayname':['Monday','Tuesday','Wednesday','Thursday','Friday','Saturday','Sunday'],
      'launch_weekday':[0,1,2,3,4,5,6]}
days=pd.DataFrame(days)
merge_weekdata=pd.merge(weekday_group,days,on='launch_weekday')
merge_weekdata
```

	launch_weekday	show_id	Dayname
0	0	628	Monday
1	1	852	Tuesday
2	2	906	Wednesday
3	3	1053	Thursday
4	4	1566	Friday
5	5	557	Saturday
6	6	569	Sunday

- As in the above table we can clearly see that Friday is the best day on which highest TV shows launched on Netflix

```
plt.figure(figsize=(10,5))
sns.barplot(x=merge_weekdata['Dayname'],y=merge_weekdata['show_id'])
plt.xlabel('Day Of the Week of movie launch')
plt.ylabel('Number of Movie launched')
plt.title('Number of Movies launched on a specific day')
plt.show()
```



- As in the above analysis we can say that the highest number of Movies launched on Friday on Netflix

Lets plot the Bivariate Analysis for TV Shows

- Analysis for TV Shows
- TV Shows:**
 - We will going to find the best week day for launch the **TV Shows** on Netflix*

Count the number of TV Show released on weekday wise

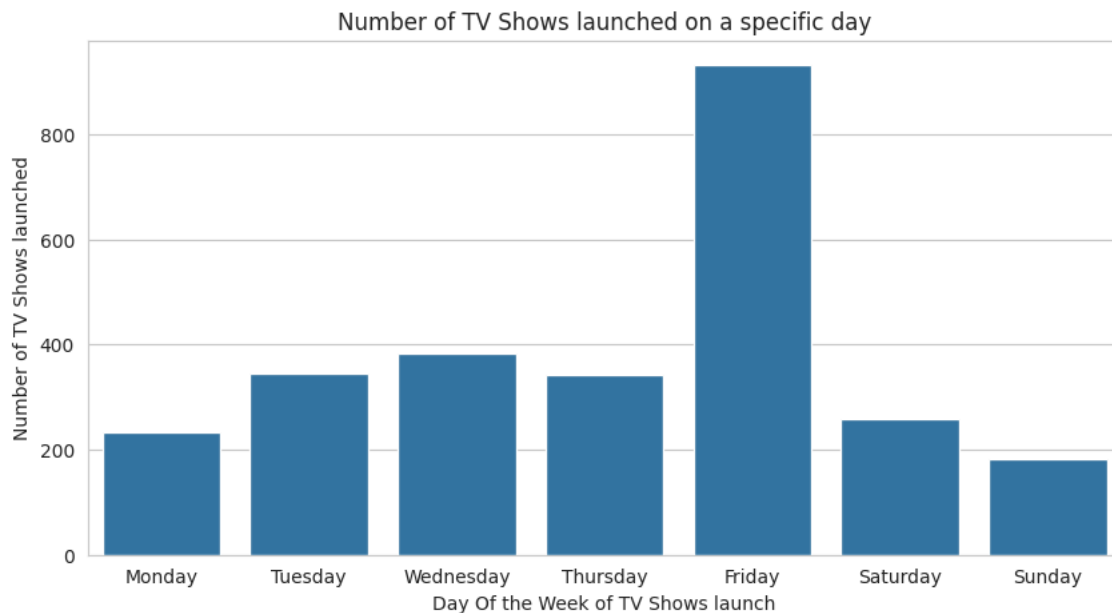
```
weekday_group_tv=tv_shows.groupby('launch_weekday')['show_id'].count().reset_index()
days={'Dayname':['Monday','Tuesday','Wednesday','Thursday','Friday','Saturday','Sunday'],
      'launch_weekday':[0,1,2,3,4,5,6]}
days=pd.DataFrame(days)
merge_weekdata_tv=pd.merge(weekday_group_tv,days,on='launch_weekday')
merge_weekdata_tv
```



	launch_weekday	show_id	Dayname
0	0	233	Monday
1	1	345	Tuesday
2	2	382	Wednesday
3	3	343	Thursday
4	4	932	Friday
5	5	259	Saturday
6	6	182	Sunday

- As in the above table we can clearly see that Friday is the best day on which highest TV shows launched on Netflix

```
plt.figure(figsize=(10,5))
sns.barplot(x=merge_weekdata_tv['Dayname'],y=merge_weekdata_tv['show_id'])
plt.xlabel('Day Of the Week of TV Shows launch')
plt.ylabel('Number of TV Shows launched')
plt.title('Number of TV Shows launched on a specific day')
plt.show()
```



- As in the above analysis we can say that the highest number of TV Shows launched on Friday on Netflix

2. Find which is the best month to release the Tv-show or the movie.

- Analysis for Tv-shows
- TV Shows:**
 - We will going to find the best month on which highest number of **TV Shows** *launched on Netflix*

Movies: As per below analysis we can say that the first month of the quarter is the best month to launch the movie

```
movie_shows.head(5)
```



	index	show_id	type	date_added	release_year	launch_day	launch_month	launch_year	launch_weekday
0	0	s1	Movie	2021-09-25	2020	25	9	2021	5
6	6	s7	Movie	2021-09-24	2021	24	9	2021	4
7	7	s8	Movie	2021-09-24	1993	24	9	2021	4
9	9	s10	Movie	2021-09-24	2021	24	9	2021	4
12	12	s13	Movie	2021-09-23	2021	23	9	2021	3

Count the number of Movies released on Month wise

```
month_group=movie_shows.groupby('launch_month')['show_id'].count().reset_index()
month_group
```



	launch_month	show_id
0	1	546
1	10	545
2	11	498
3	12	547
4	2	382
5	3	529
6	4	550
7	5	439
8	6	492
9	7	565
10	8	519
11	9	519

```
month_group['launch_month']=month_group['launch_month'].astype('int64')
```

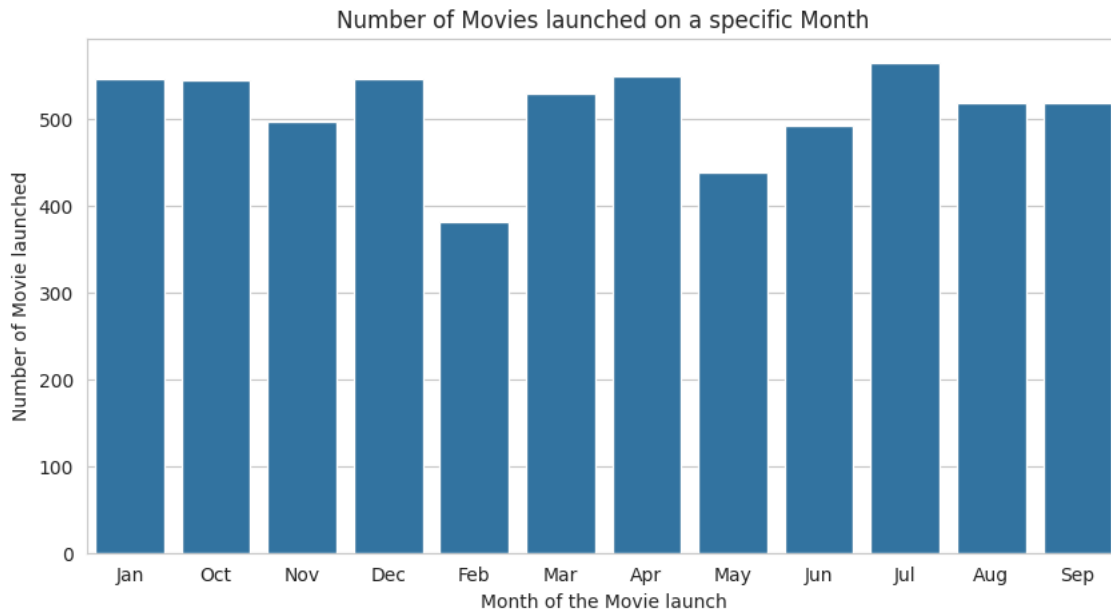
```
months={'Monthname':['Jan','Feb','Mar','Apr','May','Jun','Jul','Aug','Sep','Oct','Nov','Dec'],  
        'launch_month':[1,2,3,4,5,6,7,8,9,10,11,12]}  
months=pd.DataFrame(months)
```

```
merge_month=pd.merge(month_group,months,on='launch_month')  
merge_month
```



	launch_month	show_id	Monthname
0	1	546	Jan
1	10	545	Oct
2	11	498	Nov
3	12	547	Dec
4	2	382	Feb
5	3	529	Mar
6	4	550	Apr
7	5	439	May
8	6	492	Jun
9	7	565	Jul
10	8	519	Aug
11	9	519	Sep

```
plt.figure(figsize=(10,5))  
sns.barplot(x=merge_month['Monthname'],y=merge_month['show_id'])  
plt.xlabel('Month of the Movie launch')  
plt.ylabel('Number of Movie launched')  
plt.title('Number of Movies launched on a specific Month')  
plt.show()
```



As per above chart analysis we can see that [Jan,Mar,Apr,Jul,oct and dec] are the best month to launch the movie

TV Shows : As per below analysis we can say that the August and December months are the best Months on which the TV Shows launch

Count the number of TV Shows released on Month wise

```
month_group_tv=tv_shows.groupby('launch_month')['show_id'].count().reset_index()
month_group_tv['launch_month']=month_group_tv['launch_month'].astype('int64')

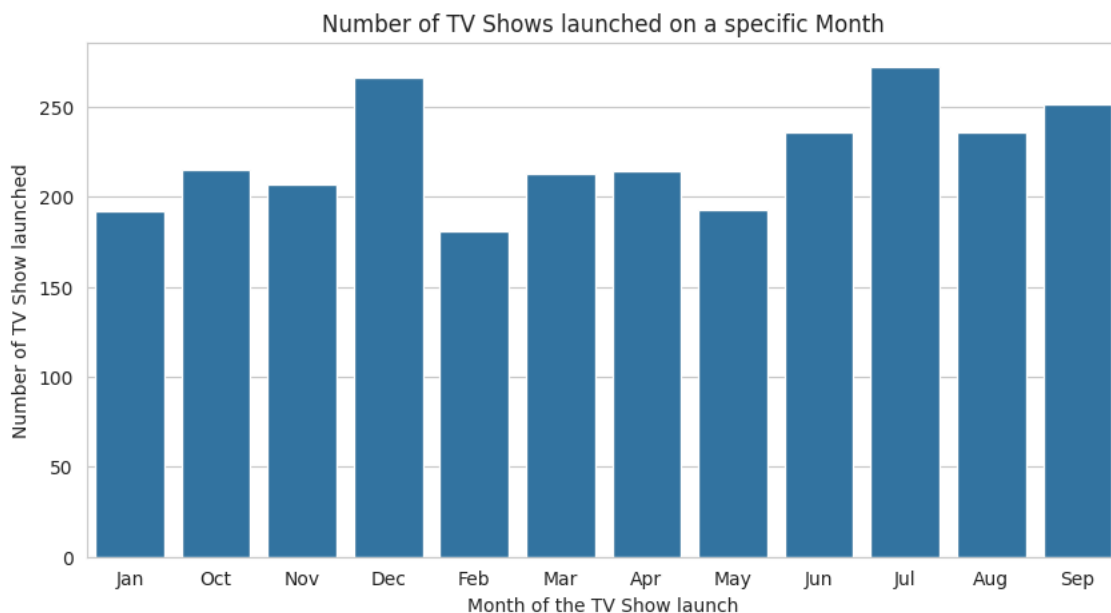
months={'Monthname':['Jan','Feb','Mar','Apr','May','Jun','Jul','Aug','Sep','Oct','Nov','Dec'],
        'launch_month':[1,2,3,4,5,6,7,8,9,10,11,12]}
months=pd.DataFrame(months)
merge_month_tv=pd.merge(month_group_tv,months,on='launch_month')
merge_month_tv
```



	launch_month	show_id	Monthname
0	1	192	Jan
1	10	215	Oct
2	11	207	Nov
3	12	266	Dec
4	2	181	Feb
5	3	213	Mar
6	4	214	Apr
7	5	193	May
8	6	236	Jun
9	7	272	Jul
10	8	236	Aug
11	9	251	Sep

- As per below graphical analysis we can say that the August and December months are the best Months on which the most number of TV Shows launch

```
plt.figure(figsize=(10,5))
sns.barplot(x=merge_month_tv['Monthname'],y=merge_month_tv['show_id'])
plt.xlabel('Month of the TV Show launch')
plt.ylabel('Number of TV Show launched')
plt.title('Number of TV Shows launched on a specific Month')
plt.show()
```

The best launch day for Movies/Shows is Friday

```
'''
plt.figure(figsize=(15,5))
sns.histplot(dayname)
plt.xlabel('dayname')
plt.ylabel('Number of Show/Movie Launched')
plt.show()
'''
```



```
\nplt.figure(figsize=(15,5))\nsns.histplot(dayname)\nplt.xlabel('dayname')\nplt.ylabel('Number of Show/Movie Launched')\nplt.show()\n'
```

✓ 10. Director has worked for how many countries

```
countplot=df[['director','country']].groupby('director').count().reset_index()
countplot.head()
```



	director	country
0	Aaron Moorhead	24
1	Aaron Woolf	2
2	Abbas Alibhai Burmawalla	18
3	Abdullah Al Noor	30
4	Abhinav Shiv Tiwari	24

✓ 11. WorldCloud : Which genre movies are more popular or produced more

```
from wordcloud import WordCloud, STOPWORDS
```

```
comment_words = ""
stopwords = set(STOPWORDS)
```

```
for val in df.description:
    val = str(val)
    tokens = val.split()
```

```
for i in range(len(tokens)):
    tokens[i] = tokens[i].lower()
```

```
comment_words += " ".join(tokens)+" "
comment_words
```

- ✓ Graphical design of worldcloud

```
plt.show()
```



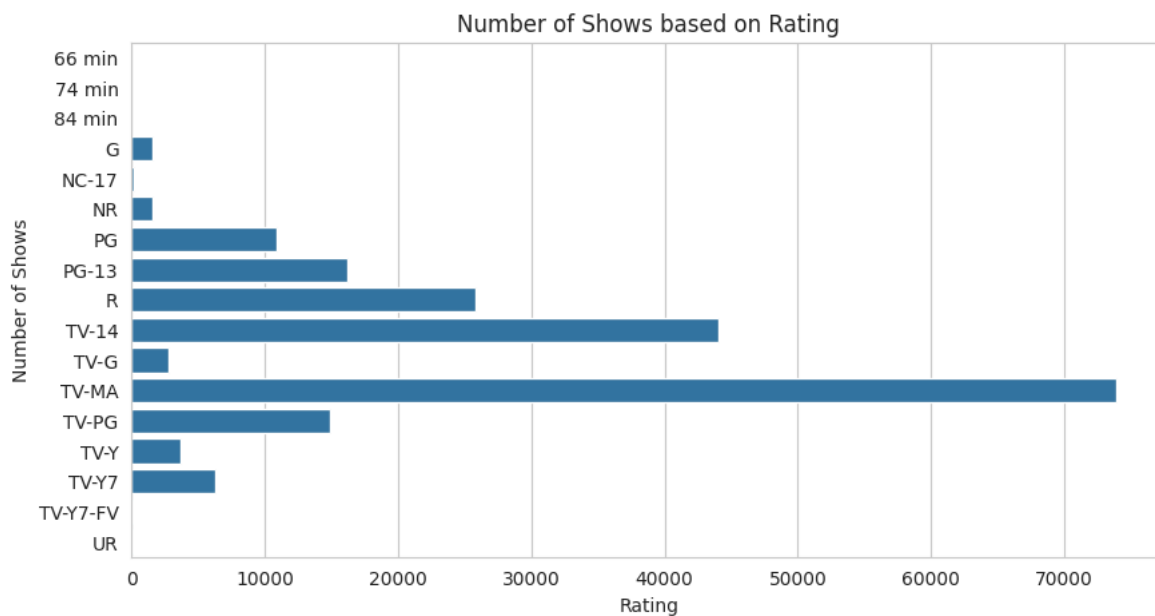
- Number of shows launched against specific rating

26/34

```
<ipython-input-358-be8214118b87>:1: FutureWarning: The default of observed=False is deprecated and will be changed to True in a future version of pandas. F
df.groupby('rating')['show_id'].count().reset_index()
```

	rating	show_id
0	66 min	1
1	74 min	1
2	84 min	1
3	G	1530
4	NC-17	149
5	NR	1573
6	PG	10919
7	PG-13	16246
8	R	25860
9	TV-14	44001
10	TV-G	2779
11	TV-MA	73938
12	TV-PG	14926
13	TV-Y	3665
14	TV-Y7	6304
15	TV-Y7-FV	86
16	UR	86

```
plt.figure(figsize=(10,5))
sns.countplot(df['rating'])
#plt.hist(df['rating'])
plt.xlabel('Rating')
plt.ylabel('Number of Shows')
plt.title('Number of Shows based on Rating')
plt.show()
```



Based on the above analysis we recommend that,

- The show with Rating 'TV MA' would be more commended to launch on Netflix.
- The Shows which has Raing 'G' and below that is not more recommended.

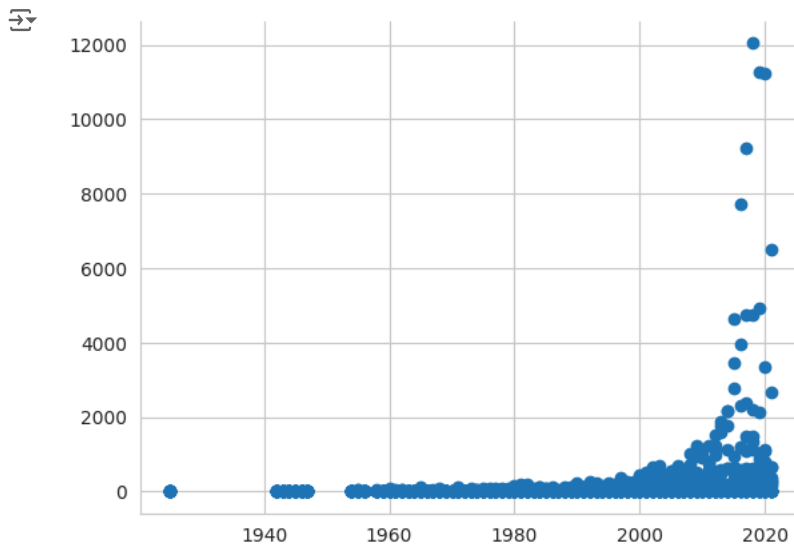
```
released_gourp_data=df.groupby(['rating','release_year'])['show_id'].count().reset_index()
released_gourp_data
```

```
<ipython-input-360-367e211a562b>:1: FutureWarning: The default of observed=False is deprecated and will be changed to True in a future version of pandas. F
released_gourp_data=df.groupby(['rating','release_year'])['show_id'].count().reset_index()
```

	rating	release_year	show_id
0	66 min	1925	0
1	66 min	1942	0
2	66 min	1943	0
3	66 min	1944	0
4	66 min	1945	0
...	...	...	...
1253	UR	2017	0
1254	UR	2018	0
1255	UR	2019	0
1256	UR	2020	0
1257	UR	2021	0

1258 rows × 3 columns

```
from matplotlib import pyplot as plt
plt.scatter(x='release_year', y='show_id', data=released_gourp_data)
plt.gca().spines[['top', 'right']].set_visible(False)
```



3. Non-Graphical Analysis: Value counts and unique attributes (10 Points)

df.columns

```
Index(['show_id', 'type', 'title', 'director', 'cast', 'country', 'date_added',
       'release_year', 'rating', 'duration', 'listed_in', 'description',
       'launch_day', 'launch_month', 'launch_year'],
      dtype='object')
```

- Find Unique values of columns

df['type'].unique()

```
['Movie', 'TV Show']
Categories (2, object): ['Movie', 'TV Show']
```

- Total number of shows for Movie and TV Shows

df['type'].value_counts().reset_index()



	type	count
0	Movie	145917
1	TV Show	56148

```
df['country'].unique()
```

```
len(df['country'].unique())
```



```
197
```

- Top 10 countries with highest movie/tvshows

```
df['country'].value_counts().sort_values(ascending=False).reset_index().head(10)
```



	country	count
0	United States	61765
1	India	22139
2	United Kingdom	9733
3	United States	9482
4	Japan	7317
5	South Korea	4628
6	Canada	4395
7	Spain	4255
8	France	4182
9	France	4072

```
df['rating'].unique() #total 18 unique values
```



```
['PG-13', 'TV-MA', 'PG', 'TV-14', 'TV-PG', ..., '84 min', '66 min', 'NR', 'TV-Y7-FV', 'UR']  
Length: 17  
Categories (17, object): ['66 min', '74 min', '84 min', 'G', ..., 'TV-Y', 'TV-Y7', 'TV-Y7-FV', 'UR']
```

- Total 10 highest Number of Movies/TV shows as per rating waise

```
df['rating'].value_counts().sort_values(ascending=False).reset_index().head(10)
```



	rating	count
0	TV-MA	73938
1	TV-14	44001
2	R	25860
3	PG-13	16246
4	TV-PG	14926
5	PG	10919
6	TV-Y7	6304
7	TV-Y	3665
8	TV-G	2779
9	NR	1573

```
df['duration'].unique() #Total 221 unique values
```



```
['90 min', '2 Seasons', '1 Season', '91 min', '125 min', ..., '228 min', '18 min', '205 min', '201 min', '191 min']  
Length: 220  
Categories (220, object): ['1 Season', '10 Seasons', '10 min', '100 min', ..., '96 min', '97 min', '98 min', '99 min']
```

- Top 10 highest number of shows duration wise

```
df['duration'].value_counts().sort_values(ascending=False).reset_index().head(10)
```



	duration	count
0	1 Season	35038
1	2 Seasons	9559
2	3 Seasons	5084
3	94 min	4343
4	106 min	4040
5	97 min	3624
6	95 min	3560
7	96 min	3511
8	93 min	3480
9	90 min	3305

df['release_year'].unique() #Total 74 unique values in this attribute



[2020, 2021, 1993, 2018, 1996, ..., 1942, 1955, 1944, 1947, 1943]
 Length: 74
 Categories (74, int64): [1925, 1942, 1943, 1944, ..., 2018, 2019, 2020, 2021]

- Top 10 highest number of Shows released year wise.

df['release_year'].value_counts().sort_values(ascending=False).reset_index().head(10)



	release_year	count
0	2018	24441
1	2019	21931
2	2017	20516
3	2020	19697
4	2016	18465
5	2015	14128
6	2021	11894
7	2014	9098
8	2013	7745
9	2012	6354

df['director'].unique() #Total 4529 unique values



array(['Kirsten Johnson', 'Noam Murro', 'Julien Leclercq', ...,
 'Majid Al Ansari', 'Peter Hewitt', 'Mozes Singh'], dtype=object)

- Top 10 directors which directed highest number of Shows.

df['director'].value_counts().sort_values(ascending=False).reset_index().head(10)



	director	count
0	Noam Murro	50862
1	Martin Scorsese	419
2	Youssef Chahine	409
3	Cathy Garcia-Molina	356
4	Steven Spielberg	355
5	Lars von Trier	336
6	Raja Gosnell	308
7	Tom Hooper	306
8	McG	293
9	David Dhawan	270

✓ 4. Visual Analysis - Univariate, Bivariate after pre-processing of the data

✓ 4.1 For continuous variable(s): Distplot, countplot, histogram for univariate analysis (10 Points)

Univariate Analysis Plots

1. Distplot

```
sns.distplot(df['release_year'])  
plt.show()
```



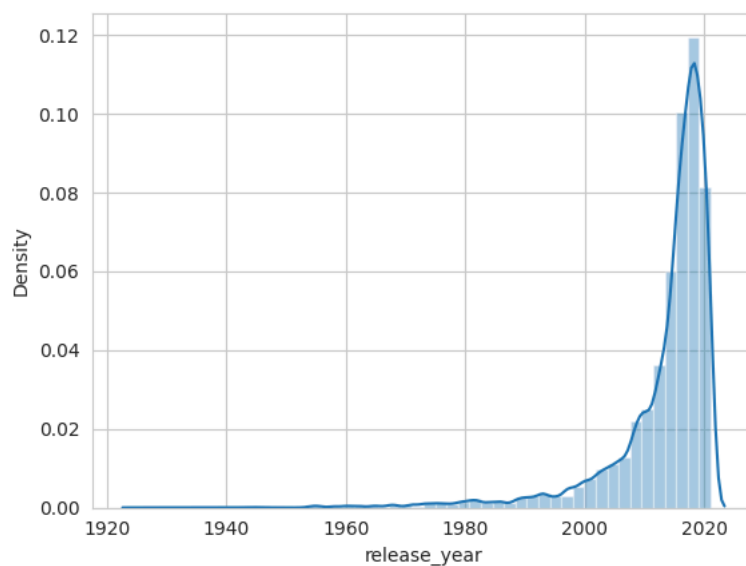
<ipython-input-376-9f1dd215b1d3>:1: UserWarning:

`distplot` is a deprecated function and will be removed in seaborn v0.14.0.

Please adapt your code to use either `displot` (a figure-level function with similar flexibility) or `histplot` (an axes-level function for histograms).

For a guide to updating your code to use the new functions, please see <https://gist.github.com/mwaskom/de44147ed2974457ad6372750bbe5751>

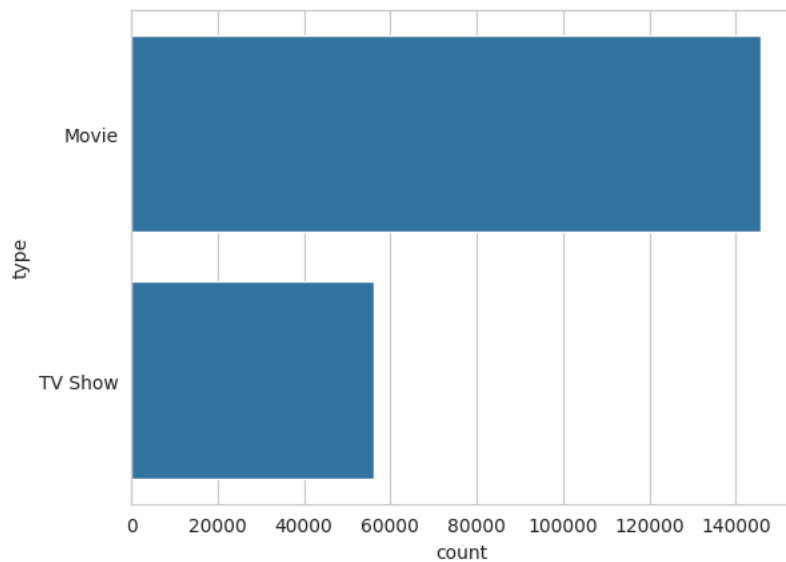
```
sns.distplot(df['release_year'])
```



- In the above plot it is showing the Density of released_year of the movies

2. Countplot

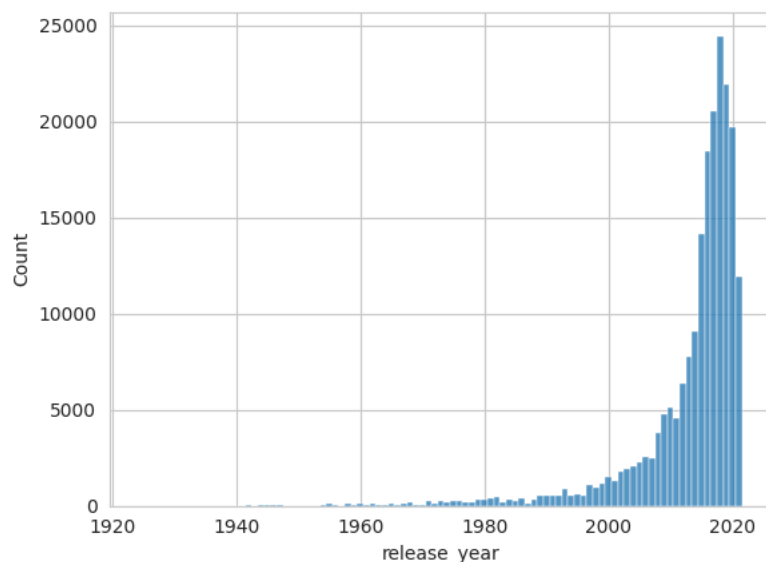
```
sns.countplot(df['type'])  
plt.show()
```



- In the above plot we can see that highest count is showing for Movie and lowest count is showing for TV Shows

3. histogram

```
sns.histplot(df['release_year'])
plt.show()
```



- In histogram plot we can find the total number of shows as per years
- As per the plot more number of shows released in the last 10 years

✓ 4.2 For categorical variable(s): Boxplot (10 Points)

- Categorical Variable Analysis

```
df.head(2)
```

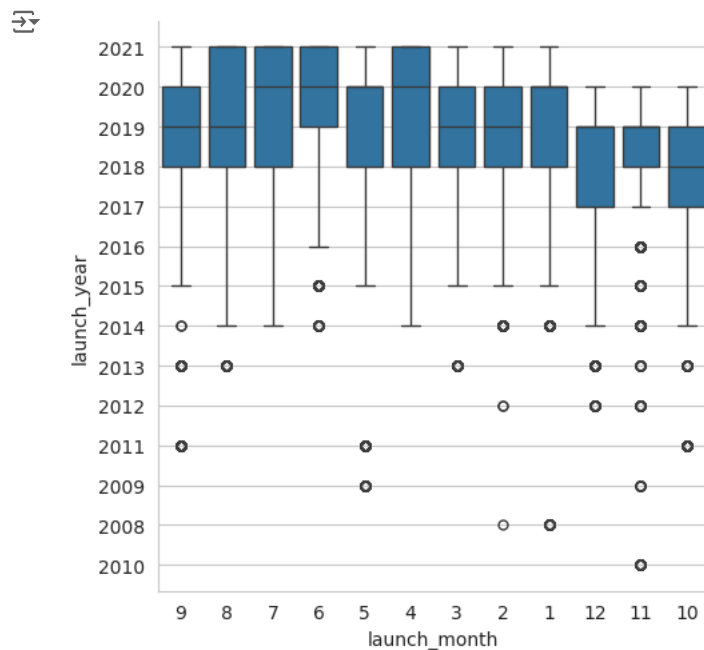


	show_id	type	title	director	cast	country	date_added	release_year	rating	duration	listed_in	description	lau
0	s1	Movie	Dick Johnson Is Dead	Kirsten Johnson	David Attenborough	United States	2021-09-25	2020	PG-13	90 min	Documentaries	As her father nears the end of his life, filmm...	After


```
l_day=df.groupby('launch_day')['show_id'].count().reset_index().head(10)
l_day
```

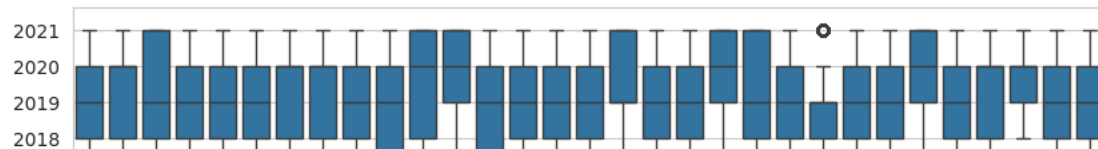
	launch_day	show_id
0	1	53594
1	10	4577
2	11	3384
3	12	3415
4	13	3421
5	14	4367
6	15	15709
7	16	8159
8	17	3565
9	18	4195

```
sns.catplot(data=df, x="launch_month", y="launch_year", kind="box")
plt.show()
```



- In the above BoxPlot it is showing that data distribution between launch years
- Box plot of month with respective of year

```
plt.figure(figsize=(10,5))
sns.set_style("whitegrid")
sns.boxplot(data=df, x="launch_day", y="launch_year")
plt.show()
```



- Boxplot of launch_day with respect to launch_year

2018

4.3 For correlation

2013

1. Heatmaps

df.columns



```
Index(['show_id', 'type', 'title', 'director', 'cast', 'country', 'date_added',
      'release_year', 'rating', 'duration', 'listed_in', 'description',
      'launch_day', 'launch_month', 'launch_year'],
      dtype='object')
```

```
core=df[['release_year','launch_day','launch_month','launch_year']]
core['release_year']= pd.to_numeric(core['release_year'])
core['launch_day']= pd.to_numeric(core['launch_day'])
```